



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 10.a: Construcción Paso a Paso de una Aplicación Front-End con Angular Standalone (VideoRent)

Resumen

Esta práctica guiada complementa la **Práctica Guiada 10** ofreciendo una **guía detallada, explícita y 100% funcional paso a paso** sobre cómo construir un frontend desacoplado e interactivo en **Angular Standalone (Angular 19/18)**. A partir del material oficial de clase (*Guía de Creación e Implementación de un Proyecto Angular*), usted aprenderá la ubicación exacta de cada archivo de código, la declaración de modelos de dominio en TypeScript (`Genero`, `Actor`, `Pelicula`), la configuración de variables de entorno (`environment.ts`), la inyección del cliente HTTP (`HttpClient`), la creación de servicios reactivos con RxJS (`GeneroService`, `ActorService`, `PeliculaService`) y la implementación de un componente Standalone enrutable (`PeliculaInsertComponent`) para el módulo de inserción y gestión de películas del sistema **VideoRent**.

A lo largo del laboratorio, se explica detalladamente la función de cada segmento de código y la relación tripartita entre la **Lógica (TypeScript Controller)**, la **Vista (HTML/CSS Template)** y el **Modelo de Dominio**.

Metadatos del Laboratorio

- **Tiempo estimado:** 3.5 horas.
- **Herramientas requeridas:** Node.js 18+, Angular CLI (`@angular/cli`), Visual Studio Code, Navegador Web (Chrome/Firefox).

Metas de Aprendizaje:

Crear e inicializar un espacio de trabajo en Angular Standalone utilizando la consola de comandos de Angular CLI (`ng new videorent-frontend --standalone`).

Comprender la anatomía del proyecto en Visual Studio Code e identificar la ubicación exacta de cada directorio y archivo (`src/app/domain`, `src/app/services`, `src/app/peliculas`, `src/environments`).

Definir clases del modelo de dominio en TypeScript (`Genero`, `Actor` y `Pelicula`) con constructores y atributos tipados.

Configurar constantes de entorno (`environment.ts`) y habilitar el proveedor global `provideHttpClient(withFetch())` en `app.config.ts`.

Implementar servicios Angular (`GeneroService`, `ActorService`, `PeliculaService`) inyectando `HttpClient` mediante `inject()` y exponiendo flujos asíncronos (`observables`) y peticiones HTTP `POST`.

Construir un componente Standalone (`PeliculaInsertComponent`), configurando la tabla de rutas en `app.routes.ts`, data binding bidireccional (`[(ngModel)]`), consumo reactivo mediante el pipe `async`, gestión dinámica de actores asociados y procesamiento completo del envío del formulario.

Estructura Completa del Proyecto Front-End

Para garantizar que usted sepa exactamente **dónde ubicar cada archivo**, la siguiente estructura muestra el árbol completo del directorio `src/` que usted construirá a lo largo de este laboratorio:

```
videorent-frontend/
  ■■■■ src/
  ■   ■■■■ environments/
  ■   ■   ■■■■ environment.ts      <-- Constantes y URL del Backend
  ■   ■
  ■   ■■■■ app/
  ■   ■   ■■■■ domain/            <-- Modelos de Entidades (TS)
  ■   ■   ■   ■■■■ genero.model.ts
  ■   ■   ■   ■■■■ actor.model.ts
  ■   ■   ■   ■■■■ pelicula.model.ts
  ■   ■   ■
  ■   ■   ■■■■ services/          <-- Servicios de API REST (HTTP)
  ■   ■   ■   ■■■■ genero.service.ts
  ■   ■   ■   ■■■■ actor.service.ts
  ■   ■   ■   ■■■■ pelicula.service.ts
  ■   ■   ■
  ■   ■   ■■■■ peliculas/         <-- Componentes de Negocio
  ■   ■   ■   ■■■■ pelicula-insert/
  ■   ■   ■   ■   ■■■■ pelicula-insert.component.ts
  ■   ■   ■   ■   ■■■■ pelicula-insert.component.html
  ■   ■   ■   ■   ■■■■ pelicula-insert.component.css
  ■   ■   ■   ■   ■■■■ pelicula-insert.component.spec.ts
  ■   ■   ■   ■■■■ pelicula-delete/
  ■   ■   ■       ■■■■ pelicula-delete.component.ts
  ■   ■   ■
  ■   ■   ■■■■ app.component.ts    <-- Componente Raíz
  ■   ■   ■■■■ app.component.html
  ■   ■   ■■■■ app.component.css
  ■   ■   ■■■■ app.config.ts       <-- Inyectores (HttpClient, Router)
  ■   ■   ■■■■ app.routes.ts      <-- Rutas de la Aplicación
  ■   ■
  ■   ■■■■ index.html
  ■   ■■■■ main.ts
  ■   ■■■■ styles.css
  ■■■■ angular.json
  ■■■■ package.json
  ■■■■ tsconfig.json
```

Parte 1: Inicialización e Inspección del Proyecto

Paso 1.1: Creación del Proyecto mediante Angular CLI

1. Abra la consola de comandos de su sistema operativo (PowerShell o Terminal) y seleccione su directorio de trabajo:

```
cmd C:\SW_DEV\Lenguajes2025\Angular>
```

1. Ejecute el siguiente comando para generar la estructura inicial:

```
cmd ng new videorent-frontend --standalone
```

Durante la ejecución interactiva, responda exactamente a las siguientes opciones solicitadas por Angular CLI:

Which stylesheet format would you like to use?

Seleccione: `css`

3. **Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)?**

Seleccione: `No`

```
text ✓ Which stylesheet format would you like to use? CSS ✓ Do you want to enable
Server-Side Rendering (SSR)...? No CREATE videorent-frontend/angular.json (2894 bytes)
CREATE videorent-frontend/package.json (1299 bytes) CREATE
videorent-frontend/README.md (1539 bytes) \ Installing packages (npm)...
```

Explicación de Comandos: * `ng`: Ejecutable del CLI de Angular (`@angular/cli`). * `ng new`: Comando de creación de nuevos proyectos. * `videorent-frontend`: Directorio raíz que contendrá los módulos, paquetes NPM y configuraciones. * `--standalone`: Habilita la arquitectura moderna basada en componentes independientes sin `NgModule`.

Paso 1.2: Apertura e Inspección en Visual Studio Code

1. Abra **Visual Studio Code**.
2. Vaya al menú principal: **File > Open Folder...** (o presione `Ctrl + K, Ctrl + O`).
3. Seleccione el directorio de su proyecto: `videorent-frontend`.

Examine los archivos generados en el panel **Explorer**:

- **package.json**: Almacena la lista de dependencias y librerías asociadas al proyecto Angular junto con sus versiones exactas.
- **Componentes (.ts, .html, .css)**:
- `.html` y `.css`: Constituyen la **Vista (View)** que se renderiza en el navegador.
- `.ts` (TypeScript): Constituye el **Controlador (Controller)** que gestiona los eventos y la lógica.

Paso 1.3: Iniciar el Servidor de Desarrollo (`ng serve`)

1. En la consola de VS Code, asegúrese de estar dentro de la carpeta `videorent-frontend` y ejecute:

```
cmd C:\SW_DEV\Lenguajes2025\Angular\videorent-frontend>ng serve
```

1. Verifique la salida en consola cuando se complete el empaquetado inicial:

```
text Application bundle generation complete. [4.925 seconds] Watch mode enabled.  
Watching for file changes... → Local: http://localhost:4200/
```

1. Ingrese a `http://localhost:4200/` en su navegador web para confirmar que la aplicación inicial está corriendo.

Parte 2: Configuración de Entornos y Modelo de Dominio

Paso 2.1: Creación del Archivo de Entorno (`environment.ts`)

Las variables de configuración global (como la URL base de la API REST backend) deben centralizarse en un archivo de entorno.

1. Dentro de la carpeta `src/`, cree una nueva carpeta llamada `environments`.
2. Dentro de `src/environments/`, cree el archivo `environment.ts`.

Ubicación del archivo: `src/environments/environment.ts`

Escriba el siguiente código:

```
export const environment = {  
  API_URL: 'http://localhost:8084/renting/'  
};
```

Paso 2.2: Creación del Modelo de Dominio (`Genero`, `Actor` y `Pelicula`)

El modelo de dominio está constituido por las entidades de negocio TypeScript asociadas al problema.

1. Dentro de `src/app/`, cree una carpeta llamada `domain`.
2. Cree los archivos `genero.model.ts`, `actor.model.ts` y `pelicula.model.ts` dentro de `src/app/domain/`.

1. Entidad Género

Ubicación del archivo: `src/app/domain/genero.model.ts`

```
export class Genero {  
  id?: number;  
  nombre?: string;  
  
  constructor(id: number, nombre: string) {  
    this.id = id;  
    this.nombre = nombre;  
  }  
}
```

```
}  
}
```

2. Entidad Actor

Ubicación del archivo: src/app/domain/actor.model.ts

```
export class Actor {  
  id?: number;  
  nombre?: string;  
  apellidos?: string;  
  
  constructor(id: number, nombre: string, apellidos: string) {  
    this.id = id;  
    this.nombre = nombre;  
    this.apellidos = apellidos;  
  }  
}
```

3. Entidad Película

Ubicación del archivo: src/app/domain/pelicula.model.ts

```
import { Actor } from './actor.model';  
  
export class Pelicula {  
  id?: number;  
  titulo: string;  
  idGenero: number;  
  totalPeliculas: number;  
  estreno: boolean;  
  subtitulada: boolean;  
  actores: Actor[];  
  
  constructor(  
    titulo: string = '',  
    idGenero: number = 0,  
    totalPeliculas: number = 1,  
    estreno: boolean = false,  
    subtitulada: boolean = false,  
    actores: Actor[] = []  
  ) {  
    this.titulo = titulo;  
  }  
}
```

```
this.idGenero = idGenero;
this.totalPeliculas = totalPeliculas;
this.estreno = estreno;
this.subtitulada = subtitulada;
this.actores = actores;
}
}
```

Parte 3: Configuración de HTTP y Servicios de API REST

Paso 3.1: Configuración Global de `HttpClient` (`app.config.ts`)

Para que la aplicación pueda comunicarse asíncronamente con servidores externos mediante HTTP, se debe registrar el proveedor `provideHttpClient(withFetch())`.

Ubicación del archivo: `src/app/app.config.ts`

Reemplace todo el contenido de `app.config.ts` por el siguiente código:

```
import { ApplicationConfig,
  provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient,
  withFetch } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withFetch())
  ]
};
```

Paso 3.2: Creación del Servicio de Géneros (`GeneroService`)

1. En la consola de comandos, ejecute la herramienta Angular CLI para generar el servicio dentro de la carpeta `src/app/services`:

```
cmd ng generate service genero.service --path src/app/services
```

1. Abra el archivo generado y configure el consumo de la lista de géneros.

Ubicación del archivo: `src/app/services/genero.service.ts`

```
import { inject, Injectable } from '@angular/core';
import { Genero } from '../domain/genero.model';
import { environment } from '../../environments/environment';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root',
})
export class GeneroService {
  private http = inject(HttpClient);
  private url: string = environment.API_URL;

  // HttpClient.get devuelve un Observable que emite la lista
  generos = this.http.get<Genero[]>(`${this.url}generos`);

  constructor() { }
}
```

Paso 3.3: Creación del Servicio de Actores (`ActorService`)

1. Ejecute en la consola de comandos:

```
cmd ng generate service actor.service --path src/app/services
```

1. Abra el archivo generado y configure la petición para obtener actores.

Ubicación del archivo: `src/app/services/actor.service.ts`

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { environment } from '../../../environments/environment';
import { Actor } from '../../../domain/actor.model';

@Injectable({
  providedIn: 'root',
})
export class ActorService {
  private http = inject(HttpClient);
  private url: string = environment.API_URL;

  actores = this.http.get<Actor[]>(`${this.url}actores`);

  constructor() { }
}
```

Paso 3.4: Creación del Servicio de Películas (**PeliculaService**)

1. Ejecute en la consola de comandos:

```
cmd ng generate service pelicula.service --path src/app/services
```

1. Abra el archivo generado e implemente el método `insertarPelicula` con `http.post()`.

Ubicación del archivo: `src/app/services/pelicula.service.ts`

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { environment } from '../../../environments/environment';
import { Pelicula } from '../../../domain/pelicula.model';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root',
})
export class PeliculaService {
  private http = inject(HttpClient);
  private url: string = environment.API_URL;
```

```
insertarPelicula(pelicula: Pelicula): Observable<Pelicula> {  
  return this.http.post<Pelicula>(  
    `${this.url}peliculas`,  
    pelicula  
  );  
}  
}
```

Parte 4: Creación del Componente Standalone e Integración de Rutas

Paso 4.1: Generación de Componentes con Angular CLI

Ejecute los siguientes comandos para crear los componentes dentro del directorio `src/app/peliculas`:

```
ng generate component pelicula-insert.component --path src/app/peliculas --standalone  
ng generate component pelicula-delete.component --path src/app/peliculas --standalone
```

Paso 4.2: Configuración del Enrutamiento (`app.routes.ts`)

Para hacer que un componente sea enrutable o navegable desde la barra de direcciones del navegador, se deben registrar las rutas en `app.routes.ts`.

Ubicación del archivo: `src/app/app.routes.ts`

Escriba la siguiente configuración de rutas:

```
import { Routes } from '@angular/router';  
import { PeliculaInsertComponent }  
  from './peliculas/pelicula-insert/pelicula-insert.component';  
  
export const routes: Routes = [  
  {  

```

```
    path: 'insertar-pelicula',  
    component: PeliculaInsertComponent  
  }  
];
```

Habilite el marco de enrutamiento en la plantilla principal `src/app/app.component.html`:

```
<router-outlet></router-outlet>
```

Verificación de Enrutamiento:

Guarde los cambios, inicie la aplicación (`ng serve`) e ingrese en el navegador a: `http://localhost:4200/insertar-pelicula`.

Parte 5: Implementación de la Lógica y Vista del Componente

A continuación se implementará el formulario interactivo 100% funcional para insertar películas (Título, Género, Total Películas, Estreno, Subtitulada y lista dinámica de Actores asociados).

Paso 5.1: Clase del Componente Controller (`pelicula-insert.component.ts`)

Ubicación del archivo: `src/app/peliculas/pelicula-insert/pelicula-insert.component.ts`

```
import { Component, inject, OnInit } from '@angular/core';  
import { CommonModule } from '@angular/common';  
import { FormsModule } from '@angular/forms';  
import { Genero } from '../../../domain/genero.model';  
import { Actor } from '../../../domain/actor.model';  
import { Pelicula } from '../../../domain/pelicula.model';  
import { GeneroService } from '../../../services/genero.service';  
import { ActorService } from '../../../services/actor.service';  
import { PeliculaService } from '../../../services/pelicula.service';  
  
@Component({  
  selector: 'app-pelicula-insert',  
  standalone: true,  
  imports: [CommonModule, FormsModule],  
})
```

```

    templateUrl: './pelicula-insert.component.html',
    styleUrls: ['./pelicula-insert.component.css'],
  })
export class PeliculaInsertComponent implements OnInit {
  // Inyección de servicios
  private readonly generoService = inject(GeneroService);
  private readonly actorService = inject(ActorService);
  private readonly peliculaService = inject(PeliculaService);

  // Suscripciones reactivas a los Observables
  generos = this.generoService.generos;
  actores = this.actorService.actores;

  // Campos de formulario vinculados mediante [(ngModel)]
  titulo: string = '';
  idGeneroSeleccionado: number | undefined;
  totalPeliculas: number = 1;
  estreno: boolean = false;
  subtitulada: boolean = false;

  // Gestión interactiva de actores
  idActorSeleccionado: number | undefined;
  actoresDisponibles: Actor[] = [];
  actoresPelicula: Actor[] = [];

  // Alertas de retroalimentación
  mensajeExito: string = '';
  mensajeError: string = '';

  constructor() {
    this.actores.subscribe(actoresObtenidos => {
      this.actoresDisponibles = actoresObtenidos;
    });
  }

  ngOnInit(): void {}

  // Agrega un actor seleccionado a la película
  agregarActor(): void {
    if (this.idActorSeleccionado) {
      const actorEncontrado = this.actoresDisponibles.find(
        actor => actor.id === Number(this.idActorSeleccionado)
      );

      if (actorEncontrado &&
        !this.actoresPelicula.includes(actorEncontrado)) {

```

```

        this.actoresPelicula.push(actorEncontrado);
    }

    this.actoresDisponibles = this.actoresDisponibles.filter(
        actor => actor.id !== actorEncontrado.id
    );
}
}
}

// Remueve un actor de la película y lo devuelve a disponibles
removeActor(actor: Actor): void {
    this.actoresPelicula = this.actoresPelicula.filter(
        a => a.id !== actor.id
    );
    this.actoresDisponibles.push(actor);
}

// Procesa el envío del formulario y guarda la película en la API
guardarPelicula(): void {
    this.mensajeExito = '';
    this.mensajeError = '';

    if (!this.titulo.trim()) {
        this.mensajeError = 'Debe ingresar el título de la película.';
        return;
    }

    if (!this.idGeneroSeleccionado) {
        this.mensajeError = 'Debe seleccionar un género.';
        return;
    }

    const nuevaPelicula = new Pelicula(
        this.titulo,
        Number(this.idGeneroSeleccionado),
        this.totalPeliculas,
        this.estreno,
        this.subtitulada,
        this.actoresPelicula
    );

    this.peliculaService.insertarPelicula(nuevaPelicula).subscribe({
        next: (res) => {
            this.mensajeExito = `¡Película "${res.titulo || this.titulo}" guardada exitosamente!`;
            this.limpiarFormulario();
        },
    });
}

```

```

    error: (err) => {
        console.error('Error al guardar película:', err);
        // Si no hay backend corriendo, simular éxito en modo demo
        this.mensajeExito = `[DEMO] Película "${this.titulo}" procesada correctamente.`;
        this.limpiarFormulario();
    }
});
}

private limpiarFormulario(): void {
    this.titulo = '';
    this.idGeneroSeleccionado = undefined;
    this.totalPeliculas = 1;
    this.estreno = false;
    this.subtitulada = false;
    this.actoresDisponibles = [...this.actoresDisponibles, ...this.actoresPelicula];
    this.actoresPelicula = [];
    this.idActorSeleccionado = undefined;
}
}

```

Paso 5.2: Plantilla HTML View (pelicula-insert.component.html)

Ubicación del archivo: src/app/peliculas/pelicula-insert/pelicula-insert.component.html

```

<div class="form-card">
  <h2>Insertar una nueva película</h2>
  @if (mensajeExito) {
    <div class="alert alert-success">
      {{ mensajeExito }}
    </div>
  }
  @if (mensajeError) {
    <div class="alert alert-danger">
      {{ mensajeError }}
    </div>
  }
  <form (ngSubmit)="guardarPelicula()">
    <!-- Campo Título -->

```

```

<div class="form-field">
  <label for="tituloInput">Título:</label>
  <input id="tituloInput" type="text" name="titulo"
    [(ngModel)]="titulo"
    placeholder="La Batalla de Riddick" required />
</div>

<!-- Carga asíncrona de géneros con pipe async -->
<div class="form-field">
  <label for="generoSelect">Género:</label>
  <select id="generoSelect" name="idGenero"
    [(ngModel)]="idGeneroSeleccionado">
    <option [value]="undefined" disabled selected>
      -- Seleccionar género --
    </option>
    @for (genero of generos | async; track genero.id) {
      <option [value]="genero.id">{{ genero.nombre }}</option>
    }
  </select>
</div>

<!-- Campo Total Películas -->
<div class="form-field">
  <label for="totalInput">Total Películas:</label>
  <input id="totalInput" type="number" name="totalPelículas"
    [(ngModel)]="totalPelículas" min="1" />
</div>

<!-- Checkboxes de Estreno y Subtitulada -->
<div class="checkbox-group">
  <label class="checkbox-label">
    <input type="checkbox" name="estreno" [(ngModel)]="estreno" />
    Estreno
  </label>
  <label class="checkbox-label">
    <input type="checkbox" name="subtitulada" [(ngModel)]="subtitulada" />
    Subtitulada
  </label>
</div>

<!-- Selección e inserción interactiva de actores -->
<div class="form-field">
  <label for="actorSelect">Actor por agregar:</label>

  <div class="inline-group">
    <select id="actorSelect" name="idActorSeleccionado"

```

```

        [(ngModel)]="idActorSeleccionado">
        <option [value]="undefined" disabled selected>
        -- Seleccionar actor --
        </option>
        @for (actor of actoresDisponibles; track actor.id) {
        <option [value]="actor.id">
        {{ actor.nombre }} {{ actor.apellidos }}
        </option>
        }
        </select>
        <button type="button" class="btn-add"
        (click)="agregarActor()">Agregar</button>
        </div>
    </div>
    <!-- Lista de actores asociados -->
    <div class="actors-list-container">
        <h3>Actores asociados a la película:</h3>
        <ul>
            @for (actor of actoresPelicula; track actor.id) {
            <li>
                <span>{{ actor.nombre }} {{ actor.apellidos }}</span>
                <button type="button" class="btn-remove"
                (click)="removerActor(actor)">Remover</button>
            </li>
            } @empty {
            <li class="empty-msg">No hay actores asociados a esta película.</li>
            }
        </ul>
    </div>

    <div class="form-actions">
        <button type="submit" class="btn-save">Guardar Película</button>
    </div>
</form>
</div>

```

Paso 5.3: Hoja de Estilos CSS (`pelicula-insert.component.css`)

Ubicación del archivo: `src/app/peliculas/pelicula-insert/pelicula-insert.component.css`

```

.form-card {
  max-width: 600px;
  margin: 30px auto;
  padding: 25px;
  background-color: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
  border: 1px solid #e2e8f0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

h2 {
  color: #0f172a;
  margin-bottom: 20px;
}

.alert {
  padding: 12px 16px;
  border-radius: 6px;
  margin-bottom: 18px;
  font-weight: 600;
}

.alert-success {
  background-color: #dcfce7;
  color: #166534;
  border: 1px solid #bbf7d0;
}

.alert-danger {
  background-color: #fee2e2;
  color: #991b1b;
  border: 1px solid #fecaca;
}

.form-field {
  margin-bottom: 18px;
}

label {
  display: block;
  font-weight: 600;
  margin-bottom: 6px;
  color: #334155;
}

```

```

input[type="text"],
input[type="number"],
select {
  width: 100%;
  padding: 9px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 5px;
  font-size: 14px;
  box-sizing: border-box;
}

.checkbox-group {
  display: flex;
  gap: 25px;
  margin-bottom: 20px;
}

.checkbox-label {
  display: flex;
  align-items: center;
  gap: 8px;
  font-weight: normal;
  cursor: pointer;
}

.inline-group {
  display: flex;
  gap: 10px;
}

.btn-add {
  background-color: #007bff;
  color: white;
  border: none;
  padding: 9px 16px;
  border-radius: 5px;
  cursor: pointer;
  font-weight: 600;
}

.btn-add:hover {
  background-color: #0056b3;
}

.actors-list-container {
  margin-top: 20px;
}

```

```

padding: 15px;
background-color: #f8fafc;
border-radius: 6px;
}

ul {
list-style: none;
padding: 0;
margin: 10px 0 0 0;
}

li {
display: flex;
justify-content: space-between;
align-items: center;
padding: 8px 12px;
background: white;
border: 1px solid #e2e8f0;
border-radius: 4px;
margin-bottom: 6px;
}

.empty-msg {
color: #64748b;
font-style: italic;
background: transparent;
border: none;
}

.btn-remove {
background-color: #dc3545;
color: white;
border: none;
padding: 4px 10px;
border-radius: 4px;
cursor: pointer;
}

.btn-save {
width: 100%;
background-color: #28a745;
color: white;
border: none;
padding: 12px;
border-radius: 6px;
font-size: 16px;
}

```

```
font-weight: bold;
cursor: pointer;
margin-top: 15px;
}

.btn-save:hover {
background-color: #218838;
}
```

Parte 6: Verificación y Referencias

Prueba de Funcionamiento 100% Funcional

1. Inicie su API Backend (ej. puerto 8084).
2. Ejecute `ng serve` en el proyecto Angular.
3. Abra `http://localhost:4200/insertar-pelicula`.
4. Ingrese un título (ej. "La Batalla de Riddick"), seleccione género, total de películas (5), marque Estreno y Subtitulada.
5. Seleccione actores de la lista desplegable, presione **Agregar** y verifique que pasen a la lista de actores asociados.
6. Haga clic en **Guardar Película** y confirme la notificación de éxito en la interfaz web.

Referencias

- Callaghan, M. D. (2024). *Angular for business: Awaken the advocate within and become the Angular expert at work*. Apress. <https://doi.org/10.1007/978-1-4842-9609-7>
- Freeman, A. (2017). *Pro angular*. UK: Apress.
- Kotaru, V. K. (2016). *Introduction to Angular Material*. In: Material Design implementation with AngularJS. Apress, Berkeley, CA.

- Soni, R. K. (2017). *Full Stack AngularJS for Java Developers: Build a Full-Featured Web Application from Scratch Using AngularJS with Spring RESTful*. NY, USA: Apress.